

MSc dissertation draft — Iscte, Instituto Universitário de Lisboa (ISCTE-IUL)

Edge Gateway Dissertation — Chapter 2 Review Extract

(public dissertation title pending supervisor decision D001)

Background and Related Work

Preliminary draft for supervisor feedback — 13 August 2026

This document is Chapter 2 of the dissertation draft, extracted as a standalone document for review; chapter, section and table numbers, and cross-references to other chapters, follow the numbering of the full draft. The literature underlying this chapter is a preliminary corpus — seed references from the project context, standards and official documentation, snowballing, and preliminary non-institutional searches. The institutional database queries prescribed by the review protocol (IEEE Xplore, ACM Digital Library, Scopus/Web of Science) have not yet been executed, and every statement about the absence of related work in this chapter is bounded accordingly.

List of Acronyms

API: Application Programming Interface

C2DTA: Consumer-Controlled Digital Twin Architecture

HTTP: Hypertext Transfer Protocol

IoT: Internet of Things

JSON: JavaScript Object Notation

MQTT: Message Queuing Telemetry Transport

OCI: Open Container Initiative

QoS: Quality of Service

RAM: Random-Access Memory

SSI: Self-Sovereign Identity

TD: Thing Description

WoT: Web of Things

CHAPTER 2

Background and Related Work

This chapter provides the technical background for the artefact and positions the dissertation against related work. It is organised topic-by-topic; a final section synthesises the comparison and restates the gap. The underlying literature search is governed by the structured scoping/narrative review protocol described in Section 3.2; only sources verified against their primary records are cited. At the time of writing, the corpus assembled under that protocol consists of seed references from the project context, standards and official documentation, and candidates identified through preliminary non-institutional searches and snowballing; the planned institutional database queries have not yet been executed, and every statement about the absence of related work in this chapter is bounded by that limitation.

2.1. Edge Computing

2.1.1. From Cloud to Edge

Edge computing designates the paradigm in which computation and storage are placed near the sources of data, at or close to the network edge, rather than exclusively in centralised data centres. Shi et al. articulate its canonical motivations: latency reduction, bandwidth economy, availability under intermittent connectivity, and the ability to keep sensitive data close to its origin [1]. Satyanarayanan traces the paradigm’s emergence from earlier work on cloudlets and proximity-based offloading, and identifies the same recurring drivers — end-to-end latency, bandwidth-induced cost, privacy, and resilience when the wide-area network is degraded — as the forces that pull computation out of the data centre [2].

For continuously sensed *personal* data these motivations compound. The telemetry emitted by consumer wearables — heart rate, skin temperature, blood-oxygen saturation, movement — is intimate by nature: from it, inferences about health, sleep, stress and daily routine are possible. A deployment model in which every reading crosses the wide-area network to a vendor-operated platform therefore maximises both the exposure surface of the data and the dependence of basic functionality on external connectivity. An edge deployment inverts this default: raw measurements are ingested, validated and materialised on infrastructure inside the household, and any outward sharing becomes a deliberate, selective act rather than a precondition of the service. This data-locality argument, together with the latency and availability arguments of the general edge literature [1], [2], is the primary motivation for the artefact studied in this dissertation.

2.1.2. Edge Gateways for Personal Sensing

Between resource-poor sensing devices and any local or remote service sits the *edge gateway*: a mains-powered, always-on device that terminates device-facing protocols, applies validation and access control, hosts local processing, and mediates any communication with the outside world. In the consumer setting the gateway is the natural place to concentrate the household’s trust: it is the single component that sees all device telemetry, and the component whose software provenance and configuration must therefore be most defensible. This reading places two engineering requirements at the centre of the present work: the gateway’s operating platform should be versioned and repeatably buildable from a pinned description rather than assembled by hand, and its services should run natively on the ARM64 architecture that dominates low-power edge hardware. Recent work has also begun to address the operational robustness of edge-hosted twin services themselves; Neagu et al., for example, propose a high-availability standby mechanism for digital-twin nodes in resource-constrained edge environments, evaluated through fault-injection experiments [3]. A dedicated pass over the edge-gateway architecture literature is part of the pending institutional search plan (review axis A, Section 3.2).

2.1.3. Benchmarking Edge Systems

Evaluating systems at the edge raises methodological questions that differ from cloud benchmarking: workloads must be representative of edge traffic patterns, the systems under test are small enough that measurement overhead can distort results, and heterogeneous hardware makes repetition discipline and statistical reporting essential. Varghese et al. survey the edge performance-benchmarking landscape and catalogue these concerns, including metric selection, workload generation and the placement of the load generator relative to the system under test [4]. Those concerns shape the experimental protocol of this dissertation (Section 3.5): the workload generator runs outside the system under test, the primary latency metric is computed from monotonic clocks inside a single process, the statistical unit is the run, and all conditions are fixed before the campaign.

2.2. Digital Twins

2.2.1. Origin and Concept

The digital-twin concept originates in product lifecycle management: Grieves and Vickers describe a virtual construct that mirrors a physical system across its life, fed by data flowing between the physical and virtual sides, and used to anticipate emergent behaviour that would otherwise surface only in operation [5]. Already in that original formulation the twin has three constituents — the physical entity, its virtual counterpart, and the data connection that keeps the two aligned — and it is the third constituent, the sustained synchronisation channel, that turns a static model into a twin. In its modern Internet of Things (IoT) reading, a digital twin is a synchronised, addressable digital counterpart of a physical entity, exposed to applications through uniform interfaces. Fuller et al. survey the enabling technologies and open research questions of the field, and note the

persistent ambiguity of the term across domains [6]. Minerva et al. provide the most systematic treatment for the IoT context, analysing definitions, application scenarios and architectural models, and characterising the twin as a continuously updated state representation whose value lies in being queryable, composable and decoupled from the physical device’s availability [7]. It is this IoT reading — the twin as a live, addressable state resource — that the present work adopts: each wearable is represented by one twin whose properties are updated from telemetry and can be read back through a stable Application Programming Interface (API).

Sustaining that reading in software implies a recurring set of platform obligations, whatever the domain: representing twin state in a structured, versionable form; giving each twin a stable identity and address; keeping state synchronised with the device under unreliable, at-least-once delivery; controlling who may read and write which parts of the state; and exposing the result through interfaces that applications can rely on. These obligations are exactly the substrate that digital-twin *platforms* exist to provide, and they set the evaluation frame for the next subsection: the question is not whether a twin can be built — it demonstrably can — but which platform obligations a given system discharges, on what infrastructure, and at what cost.

2.2.2. Digital-Twin Platforms for the IoT

Building twins directly on application code is possible, but a platform layer — managing twin state, identity, access control and interfaces — avoids re-inventing that substrate per project. The platform used by this work’s artefact is *Eclipse Ditto*, an open-source framework, hosted by the Eclipse Foundation, for managing digital twins as addressable web resources [8]. Ditto’s model organises a twin (a *thing*) into free-form *attributes* and structured *features*, each feature carrying a set of properties; things are addressed by namespaced identifiers and guarded by *policies*, fine-grained authorisation documents that bind subjects to permissions on thing and policy resources. State is manipulated through Hypertext Transfer Protocol (HTTP) and messaging APIs, including partial updates via JavaScript Object Notation (JSON) merge patch — the mechanism the ingestion controller of this work uses to fold each telemetry event into its twin (Chapter 4). Ditto is deliberately *not* an end-to-end IoT platform: device connectivity is delegated to companion components (typically a message broker), while Ditto provides the twin layer. Architecturally it is a set of microservices — gateway, policies, things, search and connectivity, over MongoDB persistence — that can be deployed selectively; this modularity is what makes a minimal-core deployment possible on resource-limited hardware. The artefact of this dissertation includes only the gateway, policies and things services, with MongoDB, on Ditto 3.9.4 [8]. Ditto also documents native Web of Things (WoT) support, introduced as an optional integration in version 2.4.0 and considered stable and active by default since version 3.0.0 [8], which connects the twin model to the standards discussed in Section 2.7.2.

Ditto is not the only option, and the choice deserves justification against alternatives representing the main platform families. *Azure Digital Twins* represents the hyperscaler platform-as-a-service family: twins are nodes in a cloud-hosted twin graph, modelled in the Digital Twins Definition Language and integrated with the vendor’s event and analytics services [9]. As vendor documentation this source is grey literature and is used here only as engineering context; the disqualifying property for the present use case is architectural, not qualitative: a cloud-hosted twin service reintroduces exactly the wide-area dependence and third-party data custody that the edge deployment is meant to remove. *WLDT* (White Label Digital Twin) represents the library family: Picone et al. provide a general-purpose library with which twins are instantiated programmatically as lightweight software processes co-located with the application [10]. This maximises flexibility and minimises footprint, but leaves state management, access control and external APIs to the adopter — precisely the substrate this project prefers to inherit from a platform. *OpenTwins* represents the composed-framework family: Robles et al. assemble an open-source stack for compositional twins with 3D visualisation and machine-learning integration, built around Eclipse Ditto as the twin core together with messaging, time-series storage and dashboard components, deployed on Kubernetes and demonstrated on an Industry 4.0 use case [11]. OpenTwins corroborates Ditto’s position as the de facto open-source twin core, while illustrating the opposite sizing philosophy: where OpenTwins adds services around Ditto, the present work subtracts them, retaining only the minimal subset that the research questions require. Self-hostability, an explicit policy model, a documented HTTP API with merge-patch semantics, availability of ARM64 container images, and the ability to run a reduced service set were the deciding criteria.

2.2.3. Digital Twins at the Edge

Most published twin deployments assume cloud hosting; a smaller body of work studies twins on constrained edge infrastructure. Picone et al. design and evaluate a modular *edge digital twin* architecture in which twins are lightweight replications of physical devices providing a digital abstraction layer close to the devices themselves, with an experimental analysis of the approach on resource-limited hardware across multiple IoT application contexts [12]. This is the closest methodological precedent for the present work’s evaluation: it demonstrates that hosting and measuring twins at the edge is feasible and worth characterising quantitatively. Complementarily, the operational-robustness line represented by Neagu et al. treats the edge twin itself as infrastructure to be hardened, contributing failover mechanisms and fault-injection evidence [3]. Neither line, within the sources examined so far, addresses the platform question studied here: what it costs, on a versioned ARM64 gateway prototype, to ingest and materialise concurrent multi-device wearable telemetry on an off-the-shelf twin platform.

2.3. Consumer Digital Twins and the C2DTA Line

Industrial digital twins serve an organisation that owns both the asset and the data. Consumer digital twins invert the ownership question: the subject of the data is a private person, and the twin aggregates measurements that are sensitive individually and more sensitive in combination. The Consumer-Controlled Digital Twin Architecture (C2DTA) of Pinto et al. addresses this setting directly: blockchain technology is used to give consumers control over their smart devices’ digital twins and the data those twins accumulate [13]. The architecture spans consumer-side gateways, twin services and distributed-ledger components for authorisation and auditability. Its implementation uses Eclipse Ditto 3.0.0 and Eclipse Mosquitto for the twin and telemetry path, integrated with ACA-Py agents, Hyperledger Fabric, Hyperledger Indy and IPFS. The paper defines eight life-cycle scenarios and evaluates the first seven across 88 documented steps. Within that broader feasibility evaluation, a dedicated smartwatch simulator emits heart-rate, geolocation and timestamp data at 1 Hz over MQTT/SSL. The deployment ran in Docker on a Proxmox/QEMU/KVM-based x86 virtual machine with 16 GB of RAM and 32 CPUs [13]. These details characterise the predecessor system; they are not a causal performance baseline for the native-ARM64 experiments in this dissertation.

Consumer control, as C2DTA construes it, has an often-implicit infrastructural precondition: control is only as strong as the custody of the twin’s primary instantiation. If the sole live copy of a person’s twins runs on third-party infrastructure, contractual and cryptographic controls still operate inside someone else’s failure and trust domain; a household-side platform gives the control mechanisms a substrate the consumer physically possesses. The maturity of that substrate is therefore not an implementation detail of the C2DTA vision but a load-bearing assumption of it — and it is precisely the assumption this dissertation sets out to engineer and measure.

This dissertation is positioned as the *edge-platform layer* of that line of work. It takes the gateway component of the C2DTA vision and defines for it a substantially broader engineering and empirical programme: a Yocto-defined, revision-pinned ARM64 operating platform; three concurrent wearable types instead of one; six workload scenarios including fault injection and a 24-hour soak; and per-container resource profiling under a pre-specified protocol, frozen before data collection. Conversely, everything above the platform layer in C2DTA — ledger, identity, marketplace and consent mechanics — is context only here: none of it is implemented or evaluated, and no result in this document supports or weakens claims about those components (Section 1.7).

2.4. Wearable Devices and Synthetic Workloads

2.4.1. Wearable Sensing and Its Telemetry

Consumer wearables span a spectrum of form factors — wrist-worn watches and bands, rings, and sensorised garments among them — and a corresponding spectrum of sensing

modalities, from optical heart-rate monitoring and skin-temperature sensing to blood-oxygen estimation and inertial motion capture; Seneviratne et al. survey the device landscape, its sensing capabilities and its systems challenges, including the prevailing communication architecture in which devices deliver data through vendor smartphone applications and cloud services rather than through open local interfaces [14]. Real deployments produce long-lived, structured, multi-signal streams: the PMData dataset, for example, logged sixteen participants over five months with a consumer smartwatch and a sports-logging application, accumulating continuous heart rate, step, distance, calorie and sleep records alongside subjective wellness reports [15]. Two properties of such telemetry matter for a gateway platform: it is *heterogeneous* (per signal type in schema and in rate) and it is *sustained* (devices emit around the clock, so the platform’s steady-state cost is the relevant cost). The rates involved also differ by modality in a systematic way: episodic wellness metrics such as skin temperature or blood-oxygen estimates are produced at intervals of seconds to minutes; heart rate is tracked effectively continuously, at aggregate rates of the order of one reading per second; and inertial motion sensing operates at raw sampling rates one to two orders of magnitude higher still, so that motion-oriented garments dominate the message budget of any gateway that serves a mixed device population [14].

The three device types modelled in this work — smartwatch, smart ring and smart clothing — were chosen to span that heterogeneity with nominal emission rates covering nearly two orders of magnitude: continuous vitals streaming at one message per second for the smartwatch (the same rate as the published C2DTA evaluation profile [13]), episodic wellness measurements at one message per five seconds for the ring, and high-rate motion and respiration telemetry at ten messages per second for the clothing, reflecting the general pattern that inertial sensing operates at rates well above episodic wellness metrics [14]. The precise schemas and rates are normative interfaces of the artefact and are specified in Chapter 4.

2.4.2. Why Synthetic Workloads

The evaluation drives the gateway with *synthetic* telemetry produced by a deterministic, seedable simulator rather than with live devices, for four reasons. First, privacy and ethics: real wearable telemetry is personal health data, and collecting it from human subjects would impose consent and data-protection obligations disproportionate to research questions that concern the platform, not the people. Second, access: as noted above, commercial wearables typically expose their data through vendor applications and clouds rather than through local interfaces a household gateway could subscribe to [14], so a faithful live pipeline would measure a vendor integration more than the platform. Third, repeatability: a seeded generator makes the workload a versioned artefact — the same seed generates the same per-device payload sequence bit for bit — which is the precondition for the repetition and comparison discipline that benchmarking methodology demands

[4]. Fourth, controllability: load sweeps, deliberately invalid payloads and device-silence windows can be injected on schedule, which no live deployment provides on demand.

The cost of this choice is value realism: synthetic measurements are drawn from plausible ranges, not from physiological models, and no claim in this work depends on the clinical meaning of the values. The platform costs under study — validation, deduplication, twin updates, resource consumption — are driven by message rate, size and schema complexity rather than by the sampled values, which is why rate fidelity was prioritised over value fidelity. This limitation and its implications are revisited with the threats to validity in Section 5.8.

2.5. Embedded Linux Build Systems for Edge Gateways

A gateway that claims to be trustworthy infrastructure should be able to answer a simple question: *exactly what software is running on it, and could an independent party rebuild it?* How its operating system is produced determines whether that question is answerable. Three families of approaches exist for putting Linux on an edge device.

The first is a *generic binary distribution* (Debian and derivatives being typical), installed from vendor images and provisioned by hand or by scripts. This is the fastest route to a working system, but the result is imperative rather than declarative: the installed system is the accumulation of package operations and manual configuration, its exact contents are not derivable from a versioned description, and two nominally identical installations diverge over time. Unused components enlarge both footprint and attack surface.

The second family is exemplified by *Buildroot*, which its manual describes as “a simple, efficient and easy-to-use tool to generate embedded Linux systems through cross-compilation” [16]. From a single configuration, Buildroot cross-compiles a toolchain, kernel and root filesystem, yielding small images with modest learning effort. Its configuration is, however, monolithic: there is no first-class layering mechanism through which board support, a middleware stack and project-specific additions can be maintained and versioned as independent, reusable units.

The third family is the *Yocto Project*, an open-source build framework in which a target image is described by declarative recipes organised into layers, evaluated by the BitBake engine [17]. Three properties of this model carry the weight of the choice made in this work. First, *composability*: hardware support, upstream package sets and project-specific customisation live in separate layers, so the gateway image is expressed as a small custom layer (**meta-egw**) on top of unmodified upstream layers. Second, *versioned rebuildability*: every input layer can be pinned to an exact revision — in this project through a **kas** manifest — so that a rebuild attempt starts from identified inputs rather than an environmental assumption. Whether a clean checkout can reconstruct the system is evaluated separately and is not established by pinning alone. Third, *longevity*: the project maintains designated long-term-support release lines; this work uses the Scarthgap (5.0) line, released in April 2024 and supported until April 2028 [17], so the platform’s base

remains maintained beyond the dissertation’s horizon. A further, operational consequence separates the image-built families from binary distributions: an image-built system is deployed and updated as a whole artefact, so the state of a device is always one of a small set of known, buildable images, whereas package-managed systems evolve in place, one package transaction at a time, into states that no one ever built or tested as a unit. For a gateway whose evaluation depends on knowing exactly what was running when a measurement was taken, the image-based model is the one that keeps the evidence chain intact. The cost is non-trivial build complexity and build time, accepted here because repeatable platform construction is evaluated under RQ1 (Section 1.4), not assumed as an incidental property. It should be noted that much of the directly comparative Yocto-versus-Buildroot material available is vendor-authored; under the review protocol it is treated as grey literature and engineering context only, and the comparison above is therefore argued from the projects’ own primary documentation [16], [17] rather than from third-party benchmarks.

2.6. Containers on Constrained ARM Hardware

The service layer of the gateway is packaged as Open Container Initiative (OCI) containers: services are packaged with their dependencies into architecture-specific images that can be pinned by digest, which extends the provenance argument from the operating system to the deployed stack. Two properties of the OCI distribution model do the work here. An image digest is a content hash, so a digest-pinned reference denotes immutable bytes — unlike a tag, which can be re-pointed silently — and a published image can carry per-architecture variants under one name, which makes “native `linux/arm64`” a verifiable property of each pinned image rather than an assumption. In the artefact, third-party service images are pinned by digest in a versioned lock file. The locally built controller still requires the complete hash-locked ARM64 Python dependency set before the service stack can be described as fully locked (Chapter 4). The viability of this choice on small ARM devices is an empirical question with an established literature. Felter et al. compare virtual machines and Linux containers on server-class hardware and establish the baseline expectation that container overhead is modest relative to full virtualisation [18]. Morabito evaluates container technologies specifically on IoT-class edge devices, including ARM single-board computers, and concludes that containerisation is viable on constrained ARM hardware [19]. More recently, Gamess and Parajuli benchmark Docker across several Raspberry Pi generations, reporting that even low-end models such as the Raspberry Pi 3 family can run a Docker engine acceptably, with limited Random-Access Memory (RAM) capacity — rather than processing overhead — the most frequently encountered constraint [20].

Two implications follow for this work. First, the memory finding is directly actionable: the twin stack includes a JSON document database and a JVM-based twin platform, so per-container RAM profiling under load is a first-class metric of the evaluation (RQ3), not an afterthought. Second, the published evidence concerns container runtimes in general,

not this stack: the resource measurements of Chapter 5 therefore characterise the cost of the specific composition used here — broker, controller, twin services and database — rather than re-deriving the general overhead results. Orchestration platforms beyond a single-node container composition (such as lightweight Kubernetes distributions) are out of scope: the artefact is a single-gateway deployment, and no research question involves multi-node scheduling.

2.7. Interoperability: MQTT and the Web of Things

2.7.1. MQTT and Telemetry Ingestion

Message Queuing Telemetry Transport (MQTT) is a lightweight publish/subscribe protocol standardised by OASIS; version 5.0 is the current OASIS Standard [21]. Its design properties — a compact wire format, hierarchical topics, decoupling of producers from consumers through a broker, and graded delivery guarantees — have made it the de facto ingestion protocol for device telemetry. The delivery semantics matter architecturally: Quality of Service (QoS) 1 (at-least-once) matches the reliability need of telemetry ingestion, but explicitly permits duplicate delivery, so a correct receiver must be idempotent. This single protocol property propagates through the entire artefact: it is why every event carries a deterministic message identifier and sequence number, and why the controller enforces run-scoped deduplication before touching any twin (Chapter 4).

The choice of QoS 1 among the standard’s three delivery levels [21] is itself an engineering trade-off. QoS 0 (at most once) silently drops messages under disturbance, which would make delivery rate — a primary metric of this evaluation — unobservable at the protocol level. QoS 2 (exactly once) removes duplicates at the transport, but at the cost of an additional acknowledgement exchange per message, and its guarantee ends at the MQTT client boundary: a crash-and-redeliver cycle between the subscribing client and the twin platform could still apply the same event twice. Since the receiving side must therefore implement application-level idempotency in any case, paying QoS 2’s per-message overhead buys little, and QoS 1 plus deterministic deduplication is the economical design — an argument that recurs in the controller design of Chapter 4.

The broker is a deployment choice with measurable consequences: Koziol et al. compare MQTT brokers for distributed edge deployments and find substantial differences in throughput and loss behaviour between implementations [22]. Mosquitto, used in this work, is attractive for a single-node edge deployment because of its small footprint and operational simplicity; broker selection is not itself under evaluation, and the experimental protocol treats the broker as a fixed component of the composition.

2.7.2. The Web of Things

Interoperability above the transport requires agreement on what a device’s data *means*. The W3C WoT Thing Description (TD) 1.1 Recommendation defines a standard, machine-readable format for describing a thing’s capabilities — its properties, actions and events, together with the protocol bindings through which they are reachable [23]. In this work

each wearable type has a TD aligned with its JSON Schema, so the twin’s data model is documented in a standard form rather than only in project-specific schemas; the two artefacts are kept consistent by construction (Chapter 4). The choice is reinforced by the platform: Eclipse Ditto documents native WoT integration, stable and active by default since version 3.0.0 [8], making the TD the natural convergence point between the wearable schemas and the twin platform’s own description of things.

2.8. Self-Sovereign Identity as Context

The broader C2DTA vision includes decentralised identity and verifiable interactions between consumer agents. In this dissertation, Self-Sovereign Identity (SSI) appears exclusively as architectural context: the evaluated platform neither implements nor depends on identity agents, ledgers or credentials, and no statement in this document should be read as an SSI or blockchain result. The delimitation is deliberate: the research questions (Section 1.4) are answerable without identity components, and the versioned project plan excludes their implementation from P0 and any unapproved contingency. They remain reference-architecture context and future work only.

2.9. Synthesis and Research Gap

The strands reviewed above supply component-level evidence for every element of the artefact: the motivations for edge placement of personal data [1], [2], the twin as an addressable state resource [7], a self-hostable twin platform [8], feasibility of twins [12] and of containers [18], [19], [20] on constrained hardware, broker behaviour at the edge [22], standardised description of device capabilities [23], versioned platform construction [17], and the consumer-control vision that frames the use case [13]. Table 2.1 summarises the works closest to this dissertation against the properties that define it.

The comparison axes of Table 2.1 are not arbitrary: each operationalises one link of the evidence chain this work aims to close. A versioned operating-system build identifies the platform’s provenance; a native ARM64 target keeps the deployment representative of edge-class hardware rather than of development machines; an off-the-shelf twin platform distinguishes reusable results from bespoke-engine results; multi-wearable ingestion makes the workload representative of a household rather than of a single-device demonstration; and a pre-specified protocol, frozen before data collection and published with its artefacts, is what allows any of the resulting numbers to be checked. The pattern across the rows is consistent: each related system contributes one or two of the defining properties, and none of the examined works combines a versioned, rebuild-tested operating platform, a native ARM64 target, an off-the-shelf twin platform and concurrent multi-wearable ingestion under a pre-specified evaluation protocol. In summary: component-level evidence exists for edge digital twins [12], broker behaviour [22] and container overhead on constrained devices [18], [19], [20], and the C2DTA line provides the architectural vision [13]; but, among the sources examined in this preliminary review — assembled from seed references and snowballing, prior to execution of the search protocol specified in Chapter 3 — no

TABLE 2.1. Closest related systems compared with this work. “n/r” = not reported in the cited publication; cells describe only what the cited sources state. The last row states the *design targets* of this work: each entry is a design intention whose demonstration is pending until the evaluation of Chapter 5 has been executed; no entry in that row is a demonstrated result.

System	OS build	Edge target	Twin platform	Wearable workload	Evaluation discipline
C2DTA [13]	Not addressed	x86 virtual machine	Eclipse Ditto 3.0.0 and Mosquitto, integrated with identity, ledger and storage services	Eight lifecycle scenarios; first seven evaluated in 88 steps; sensor path: one smartwatch at 1 Hz	Published x86-VM feasibility evaluation; not a causal ARM64 baseline
Edge DT architecture [12]	Not addressed	Constrained devices; ISA n/r	WLDT edge twins [10]	Generic IoT devices, not wearables	Open-source library; pre-specification n/r
OpenTwins [11]	No (Kubernetes on existing hosts)	n/r	Ditto core plus messaging, storage, dashboards	Industry 4.0 use case, not wearables	Open-source framework; pre-specification n/r
ODT failover [3]	n/r	Constrained edge; ISA n/r	Custom standby-twin mechanism	IoT sensor networks, not wearables	Fault-injection experiments; artefacts n/r
This work (design targets)	Design target: Yocto Scarthgap, pinned kas manifest (re-build evidence pending)	Design target: emulated ARM64 check, native ARM64 measurement (evaluation pending)	Eclipse Ditto 3.9.4, minimal core (design choice)	Design target: three concurrent device types, six scenarios (evaluation pending)	Pre-specified protocol, frozen before data collection; versioned artefacts; campaign pending

Column key: *OS build* — whether the operating platform is rebuilt from a versioned, pinned description; *Edge target* — deployment hardware class and instruction-set architecture (ISA); *Twin platform* — the digital-twin service layer used; *Wearable workload* — concurrent ingestion of telemetry from multiple wearable-device types; *Evaluation discipline* — whether the evaluation protocol is pre-specified (frozen before data collection) and its artefacts published.

published study was identified that composes these elements into a versioned, Yocto-built ARM64 gateway with a pre-specified multi-scenario evaluation of wearable digital-twin ingestion. That composition is the subject of the remaining chapters, and the claim remains deliberately bounded: it is relative to the sources examined so far, and will be re-stated against the full protocol once the institutional database searches have been executed.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016. DOI: 10.1109/JIOT.2016.2579198
- [2] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9
- [3] M. Neagu, C. M. Serban, A. Hangan, and G. Sebestyen, “Digital twins at the edge: A high-availability framework for resilient data processing in IoT sensor networks,” *Future Internet*, vol. 18, no. 3, p. 137, 2026, Article 137. DOI: 10.3390/fi18030137
- [4] B. Varghese et al., “A survey on edge performance benchmarking,” *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–33, 2021. DOI: 10.1145/3444692
- [5] M. Grieves and J. Vickers, “Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems,” in *Transdisciplinary Perspectives on Complex Systems*, Springer International Publishing, 2017, pp. 85–113. DOI: 10.1007/978-3-319-38756-7_4
- [6] A. Fuller, Z. Fan, C. Day, and C. Barlow, “Digital twin: Enabling technologies, challenges and open research,” *IEEE Access*, vol. 8, pp. 108 952–108 971, 2020. DOI: 10.1109/ACCESS.2020.2998358
- [7] R. Minerva, G. M. Lee, and N. Crespi, “Digital twin in the IoT context: A survey on technical features, scenarios, and architectural models,” *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1785–1824, 2020. DOI: 10.1109/JPROC.2020.2998530
- [8] Eclipse Foundation. “Eclipse ditto: Open source framework for digital twins in the IoT.” Project documentation; release notes for version 3.9.4 (16 July 2026) under the same site, Accessed: Aug. 7, 2026. [Online]. Available: <https://eclipse.dev/ditto/>
- [9] Microsoft Corporation. “What is Azure Digital Twins?” Azure Digital Twins documentation overview. Vendor documentation (grey literature); used for engineering context only, Accessed: Aug. 8, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/digital-twins/overview>
- [10] M. Picone, M. Mamei, and F. Zambonelli, “WLDT: A general purpose library to build IoT digital twins,” *SoftwareX*, vol. 13, p. 100 661, 2021, Article 100661. DOI: 10.1016/j.softx.2021.100661
- [11] J. Robles, C. Martín, and M. Díaz, “OpenTwins: An open-source framework for the development of next-gen compositional digital twins,” *Computers in Industry*, vol. 152, p. 104 007, 2023, Article 104007. DOI: 10.1016/j.compind.2023.104007

- [12] M. Picone, M. Mamei, and F. Zambonelli, “A flexible and modular architecture for edge digital twin: Implementation and evaluation,” *ACM Transactions on Internet of Things*, vol. 4, no. 1, pp. 1–32, 2023. DOI: 10.1145/3573206
- [13] F. Pinto, C. Ferreira da Silva, S. Moro, and P. Aquino, “Consumer-controlled digital twin architecture: How blockchain technology gives consumers control over their smart devices’ digital twins and data,” *Blockchain: Research and Applications*, vol. 7, no. 3, p. 100342, 2026, Article 100342. Available online 2025. DOI: 10.1016/j.bcra.2025.100342
- [14] S. Seneviratne et al., “A survey of wearable devices and challenges,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2573–2620, 2017. DOI: 10.1109/COMST.2017.2731979
- [15] V. Thambawita, S. A. Hicks, H. Borgli, H. K. Stensland, et al., “PMData: A sports logging dataset,” in *Proceedings of the 11th ACM Multimedia Systems Conference (MMSys ’20)*, ACM, 2020, pp. 231–236. DOI: 10.1145/3339825.3394926
- [16] The Buildroot developers. “The Buildroot user manual.” Official project documentation, Buildroot 2026.05, Accessed: Aug. 8, 2026. [Online]. Available: <https://buildroot.org/downloads/manual/manual.html>
- [17] Yocto Project. “Yocto project documentation (Scarthgap, 5.0 LTS).” Scarthgap 5.0 released April 2024; Long-Term Support until April 2028, Accessed: Aug. 7, 2026. [Online]. Available: <https://docs.yoctoproject.org/scarthgap/>
- [18] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, “An updated performance comparison of virtual machines and Linux containers,” in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2015, pp. 171–172. DOI: 10.1109/ISPASS.2015.7095802
- [19] R. Morabito, “Virtualization on Internet of Things edge devices with container technologies: A performance evaluation,” *IEEE Access*, vol. 5, pp. 8835–8850, 2017. DOI: 10.1109/ACCESS.2017.2704444
- [20] E. Gameass and M. Parajuli, “Performance evaluation of the Docker technology on different Raspberry Pi models,” in *Proceedings of the 2023 5th International Electronics Communication Conference (IECC 2023)*, ACM, 2023, pp. 27–37. DOI: 10.1145/3616480.3616485
- [21] OASIS. “MQTT version 5.0.” OASIS Standard, 7 March 2019. Edited by A. Banks, E. Briggs, K. Borgendale and R. Gupta, Accessed: Aug. 7, 2026. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>
- [22] H. Koziolok, S. Grüner, and J. Rückert, “A comparison of MQTT brokers for distributed IoT edge computing,” in *Software Architecture (ECSA 2020), Lecture Notes in Computer Science*, Springer, 2020, pp. 352–368. DOI: 10.1007/978-3-030-58923-3_23

- [23] World Wide Web Consortium (W3C). “Web of things (WoT) thing description 1.1.” W3C Recommendation, 5 December 2023, Accessed: Aug. 7, 2026. [Online]. Available: <https://www.w3.org/TR/wot-thing-description11/>